Tutorial: conta no GitHub e Git no terminal Linux

Este guia prepara o seu computador para trabalhar no repositório telemetria-cria. Ao final você terá uma conta no GitHub, o Git configurado com o seu nome e uma cópia do projeto pronta para receber código.

Tempo estimado: 30 minutos. Cada comando deve ser digitado no terminal e confirmado com Enter. Linhas que começam com `#` são comentários, não precisam ser digitadas.

As seções 1 a 8 são para os alunos. A seção 9 é só para o professor, que libera o acesso ao repositório.

1. Criar a conta no GitHub

1. Abra https://github.com/signup no navegador.
2. Informe um e-mail que você acessa com frequência, crie uma senha forte e escolha um nome de usuário. Dica: use um nome simples e profissional, por exemplo `maria-silva` ou `joaopedro-ete`. Esse nome aparece em tudo o que você publicar.
3. Confirme o código que o GitHub envia por e-mail.
4. Entre em https://github.com/settings/profile e preencha o nome completo.
5. Envie o seu nome de usuário ao professor. Ele vai adicionar você como colaborador do repositório, e você receberá um convite por e-mail. Aceite o convite.

2. Instalar o Git

Abra o terminal (atalho `Ctrl + Alt + T` na maioria das distribuições) e digite:

```
sudo apt update
sudo apt install git
git --version
```

O último comando deve mostrar algo como `git version 2.43.0`. Se mostrar, o Git está instalado.

3. Dizer ao Git quem você é

Cada commit leva o nome e o e-mail de quem o fez. Use o mesmo e-mail da conta do GitHub, senão as suas contribuições não aparecem no seu perfil.

```
git config --global user.name "Seu Nome Completo"
git config --global user.email "seu-email@exemplo.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Confira o resultado:

```
git config --global --list
```

4. Conectar o terminal à sua conta do GitHub

O GitHub não aceita a senha da conta para enviar código pelo terminal. A forma mais simples de autorizar o seu computador é pelo **GitHub CLI** (`gh`): ele mostra um código no terminal, abre o navegador e você só digita esse código.

Primeiro, entre na sua conta do GitHub no navegador (https://github.com/login). Deixe o navegador aberto.

Instale o `gh`:

```
sudo apt install gh
gh --version
```

Se o `apt` não encontrar o pacote, instale pelo repositório oficial do GitHub:

```
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg | sudo tee
/etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-
archive-keyring.gpg] https://cli.github.com/packages stable main" | sudo tee
/etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update
sudo apt install gh
```

Agora faça o login:

```
gh auth login
```

O `gh` faz algumas perguntas. Use as setas do teclado para escolher e Enter para confirmar:

1. *Where do you use GitHub?* Escolha **GitHub.com**.
2. *What is your preferred protocol for Git operations?* Escolha **HTTPS**.
3. *Authenticate Git with your GitHub credentials?* Responda **Yes**. Isso faz o `git push` usar o login automaticamente.
4. *How would you like to authenticate?* Escolha **Login with a web browser**.

O terminal mostra então um código de oito caracteres, parecido com `A1B2-C3D4`, e a mensagem *Press Enter to open github.com in your browser*. Anote o código e pressione Enter. O navegador abre a página https://github.com/login/device. Digite o código, clique em **Continue** e depois em **Authorize GitHub**. Volte ao terminal: ele deve mostrar *Logged in as* seguido do seu nome de usuário.

Se o navegador não abrir sozinho, abra você mesmo o endereço https://github.com/login/device e digite o código.

Confira quando quiser:

```
gh auth status
```

5. Baixar o repositório

Escolha uma pasta para os seus projetos e clone o repositório:

```
mkdir -p ~/Projetos
cd ~/Projetos
git clone https://github.com/ErlyAquino/telemetria-cria.git
cd telemetria-cria
ls
```

Você deve ver o `README.md` e as pastas `firmware`, `gateway`, `backend`, `frontend` e `docs`. Leia o README antes de continuar.

6. O fluxo de trabalho do dia a dia

No projeto, ninguém escreve direto no branch `main`. Cada tarefa ganha um branch próprio, e o código entra no `main` por um pull request revisado por outra pessoa.

6.1 Antes de começar: atualize o main

```
cd ~/Projetos/telemetria-cria
git checkout main
git pull
```

6.2 Crie um branch para a sua tarefa

O nome do branch descreve a tarefa, sem espaços e sem acentos:

```
git checkout -b leitura-dht22-arduino
```

6.3 Escreva o código e salve os commits

Edite os arquivos no editor que preferir. Depois, veja o que mudou e registre:

```
git status                        # lista os arquivos alterados
git add firmware/arduino-serial/ # adiciona a pasta (ou um arquivo) ao commit
git commit -m "Adiciona leitura do DHT22 pela serial"
```

Faça commits pequenos e frequentes, com mensagem curta em português dizendo o que mudou. Não coloque senhas nem chaves de Wi-Fi em nenhum arquivo: elas vão no `.env`, que o Git ignora.

6.4 Envie o branch ao GitHub

```
git push -u origin leitura-dht22-arduino
```

Nas próximas vezes, no mesmo branch, basta `git push`.

6.5 Abra o pull request

1. Entre em https://github.com/ErlyAquino/telemetria-cria. O GitHub mostra um aviso com o seu branch e o botão **Compare & pull request**. Clique nele.
2. No título, diga o que o código faz. Na descrição, diga como você testou.
3. Clique em **Create pull request**.
4. Outra pessoa da equipe revisa, pede ajustes se precisar, e aprova. O professor faz o merge.

Se pedirem alterações, edite os arquivos, faça novos commits e dê `git push`. O pull request atualiza sozinho.

6.6 Depois do merge

Volte ao `main`, atualize e apague o branch que já entrou:

```
git checkout main
git pull
git branch -d leitura-dht22-arduino
```

7. Comandos para consultar quando esquecer

Quero...	**Comando**
Ver em que branch estou e o que mudou	`git status`
Ver o histórico	`git log --oneline`

Ver a diferença antes de commitar	`git diff`
Trocar de branch	`git checkout nome-do-branch`
Listar branches	`git branch`
Desfazer alterações de um arquivo ainda não commitado	`git checkout -- arquivo`
Tirar um arquivo do próximo commit	`git reset arquivo`
Buscar as novidades do GitHub	`git pull`

8. Problemas comuns

Pede usuário e senha ao dar push, ou dá `Permission denied`. O login da seção 4 não foi feito, ou foi feito em outro computador. Rode `gh auth status`. Se aparecer *not logged in*, rode `gh auth login` de novo. Se você ainda não foi adicionado como colaborador, o push também é recusado: confira com o professor.

Erro "Please tell me who you are" ao commitar. A seção 3 não foi feita. Rode os dois comandos `git config --global` de novo.

Erro "Updates were rejected" ao dar push. Alguém enviou commits ao mesmo branch antes de você. Rode `git pull`, resolva conflitos se houver, e tente o push de novo.

Conflito de merge. O Git marca no arquivo os trechos em conflito entre `<<<<<<<` e `>>>>>>>`. Edite o arquivo deixando só a versão correta, apague as marcas, depois `git add arquivo` e `git commit`. Se travar, chame alguém da equipe: conflito é normal e se resolve em dupla.

Commitei uma senha por engano. Avise o professor imediatamente e troque a senha. Apagar o arquivo em um commit novo não remove o histórico.

9. Para o professor: liberar o acesso dos alunos

Esta seção é só para quem administra o repositório. Os alunos não precisam fazer nada aqui. Sem este passo, o `git push` do aluno falha com `Permission denied` mesmo com o login feito.

9.1 Adicionar um aluno como colaborador pelo site

1. Abra https://github.com/ErlyAquino/telemetria-cria/settings/access.
2. Clique em **Add people**.
3. Digite o nome de usuário que o aluno enviou e selecione o perfil correto na lista.
4. Deixe a permissão como **Write**. Ela permite criar branches, enviar commits e abrir pull requests, mas não apagar o repositório nem mudar as configurações.
5. Clique em **Add ... to this repository**. O aluno recebe um convite por e-mail e precisa aceitá-lo. O convite expira em 7 dias.

Para conferir quem já aceitou, volte à mesma página. Convites pendentes aparecem em uma lista separada, acima dos colaboradores ativos.

9.2 Adicionar vários alunos pelo terminal

Com o GitHub CLI (`gh`) instalado e autenticado, um comando por aluno:

```
gh api -X PUT repos/ErlyAquino/telemetria-cria/collaborators/NOME-DO-ALUNO -f
permission=push
```

Para uma turma inteira, coloque os nomes em um arquivo `alunos.txt`, um por linha, e rode:

```
while read aluno; do
  gh api -X PUT "repos/ErlyAquino/telemetria-cria/collaborators/$aluno" -f
permission=push
done < alunos.txt
```

Para listar quem tem acesso:

```
gh api repos/ErlyAquino/telemetria-cria/collaborators --jq '.[].login'
```

Para remover alguém, ao final do projeto ou se um aluno sair:

```
gh api -X DELETE repos/ErlyAquino/telemetria-cria/collaborators/NOME-DO-ALUNO
```

Se o `gh` não estiver instalado, siga a seção 4 deste tutorial.

9.3 Proteger o branch main

Para garantir que o código só entre no `main` por pull request revisado, mesmo por descuido de quem tem permissão de escrita:

1. Abra https://github.com/ErlyAquino/telemetria-cria/settings/branches.
2. Clique em **Add branch ruleset** (ou **Add classic branch protection rule**).
3. Em *Ruleset name*, escreva `protege-main`. Em *Enforcement status*, marque **Active**.
4. Em *Target branches*, adicione o `main`.
5. Marque **Require a pull request before merging** e, dentro dela, **Required approvals: 1**.
6. Marque **Block force pushes**.
7. Salve.

A partir daí, um `git push` direto no `main` é recusado, e todo pull request precisa de uma aprovação antes do merge. O próprio professor também passa a seguir o fluxo, o que serve de exemplo para a turma.

9.4 Checklist de início de turma

- [ ] Todos os alunos enviaram o nome de usuário do GitHub.
- [ ] Todos foram adicionados como colaboradores e aceitaram o convite.
- [ ] Cada aluno rodou `gh auth status` e viu *Logged in* no seu computador.
- [ ] Cada aluno clonou o repositório e abriu o README.
- [ ] O branch `main` está protegido.
- [ ] Primeira tarefa de teste: cada aluno cria um branch, adiciona o próprio nome em um arquivo `docs/equipe.md` e abre um pull request. Serve para treinar o fluxo completo antes de mexer em código.